CS6.302 — Software System Development
Lab Activity 2: Stored Procedures & Cursors
Time-Travel Analytics for a Food-Delivery Platform

Total Marks	Duration	Mode	Submission
20	1.5 hours	MySQL Workbench	Moodle only

Problem Statement

Compute metrics on time-travel data for a food-delivery application. First, load the supplied event-log dataset into a staging table. Then write stored procedures to compute the required delivery-level and requestor-level analytics into the target tables below.

Assumptions: All assumptions are open to your interpretation. Clearly state every assumption in `readme.md` and apply it consistently.

Submission Instructions

1. Provide one `.sql` file containing the required `CREATE TABLE` and `CREATE PROCEDURE` statements.
2. Provide a `readme.md` containing all assumptions, explanations/rationale, and the GitHub repository URL.
3. All submissions are accepted via Moodle only .

Dataset

Use `delivery_data_100k_expanded_pins.csv`. The generated dataset contains **100,000 unique orders** and approximately **829,801 event rows**.

Dataset URL: `https://github.com/serc-iiith/serc-iiith.github.io/blob/main/CourseContent/CS6302_M26/Labs/lab02/delivery_data_100k_expanded_pins.csv.zip`

The columns are:

Column	Meaning
OrderRequestorID	Customer/requestor ID
OrderID	Order identifier; repeats across the order lifecycle
PartnerID	Delivery partner assigned to the order
PINCode	Delivery-area PIN code
Status	Status entered by the order at the given timestamp
Timestamp	Time at which the order entered the status

Create a staging table whose columns mirror the CSV and load the data before performing the

analysis.

1. Target Table — `deliverystatistics` (6 marks)

Create a target table named `deliverystatistics`, grouped by **PINCode, PartnerID, MonthofOrder, and YearofOrder**.

Use the order's first `PendingAssignment` timestamp as the order-placement time for determining `MonthofOrder` and `YearofOrder`.

The table must contain the following columns:

Required column	Definition
`PINCode`	Grouping key; delivery-area PIN.
`PartnerID`	Grouping key; attributed delivery partner.
`MonthofOrder`	Month of the first PendingAssignment event.
`YearofOrder`	Year of the first PendingAssignment event.
`TotalOrders`	Count of distinct OrderIDs in the grouping bucket.
`TotalPendingAssignment`	Count of PendingAssignment events.
`TotalAccepted`	Count of Accepted events.
`TotalHeadingforPickup`	Count of HeadingforPickup events.
`TotalArrivedatPickup`	Count of ArrivedatPickup events.
`TotalPickedUp`	Count of PickedUp events.
`TotalOutforDelivery`	Count of OutforDelivery events.
`TotalArrivedatDoorStep`	Count of ArrivedatDoorStep events.
`TotalDelivered`	Count of Delivered events.
`TotalDropped`	Count of Dropped events.
`TotalDelayedatPickup`	Count of DelayedatPickup events.
`TotalDeliveryFailed`	Count of DeliveryFailed events.
`TotalReturningtoStore`	Count of ReturningtoStore events.
`TotalReturned`	Count of Returned events.
`TotalCancelled`	Count of Cancelled events.
`TimetoAccept`	Average minutes from PendingAssignment to the following Accepted event, per order.
`TimetoPickup`	Average minutes from Accepted to PickedUp, per order.
`TimetoArriveatDoorStep`	Average minutes from PickedUp to ArrivedatDoorStep, per order.
`TimetoDeliver`	Average minutes from the first PendingAssignment to Delivered, per order.

2. Populate `deliverystatistics` using a Stored Procedure (8 marks)

Write a stored procedure named `PopulateDeliveryStatistics()` that reads from the staging table and populates `deliverystatistics`.

Requirements:

- Use the `DELIMITER` pattern covered in the lecture.
- Make the procedure safely re-runnable by truncating or upserting the target data at the start.

- The procedure must contain at least one **explicit cursor**.
- Use a `DECLARE ... CONTINUE HANDLER FOR NOT FOUND` loop.
- Use the cursor for sequence-dependent time metrics where appropriate.
- Status-count metrics may be computed using set-based SQL such as `GROUP BY`; a cursor is not required for every metric.
- Use `TIMESTAMPDIFF` or an equivalent MySQL approach for minute-based time differences.

For dropped-and-reassigned orders, decide whether metrics are attributed to the original partner, final partner, or another consistent rule. Document the choice in `readme.md`.

3. Design and Populate `requestorstatistics` (4 marks)

Design a second target table named `requestorstatistics` using the KPIs in `deliverystatistics` as inspiration. The exact schema is intentionally open-ended.

Write a stored procedure named `PopulateRequestorStatistics()` that populates the table from the underlying event data.

Possible KPIs include:

- `TotalOrdersPlaced`
- `TotalCancelled`
- `TotalDeliveryFailed`
- `CancellationRate`
- `FailureRate`
- `AvgTimeToDeliver`
- `MostUsedPIN`
- `MostFrequentPartnerID`

These are suggestions, not a mandatory list. Select a coherent set of KPIs and explain why each KPI is useful to a food-delivery operations team.

4. Cursor vs. Set-Based Processing — `readme.md` (2 marks)

Choose one metric implemented using a cursor. In 3–5 sentences, explain whether it could instead be implemented using a set-based query such as a self-join or window function, and what would be gained or lost.

Important Data Considerations

- `PartnerID` is an empty string, not NULL, during `PendingAssignment`.

- A dropped-and-reassigned order can contain multiple `PartnerID`s and multiple `Accepted` events; document your attribution rule.
- Prototype on a small subset of OrderIDs before running over the full dataset.
- Clearly stated, reasonable assumptions will not be penalized; undocumented or inconsistent assumptions may affect evaluation.

Deliverables

File	Required contents
`solution.sql`	All CREATE TABLE and CREATE PROCEDURE statements in executable order.
`readme.md`	Assumptions, requestor KPI rationale, cursor-vs-set-based discussion, and GitHub repository URL.

Grading Rubric

Component	Marks	Assessment
Task 1 — `deliverystatistics` DDL	6	Grouping keys, all required columns, sensible types and definitions.
Task 2 — `PopulateDeliveryStatistics()`	8	Status counts, time metrics, explicit cursor and NOT FOUND handler.
Task 3 — `requestorstatistics`	4	Schema, population procedure, and KPI rationale.
Task 4 — Cursor vs. set-based	2	Clear 3–5 sentence technical comparison.

Happy Programming!